05-1-workload-fixed

accounts-api security assessment · generated from the repository, not hand-edited

```
# 05-1-workload-fixed.yaml
#
# Section 5 deliverable: the remediated Deployment and RBAC. Defects and their
# exploitability ranking are in 05-6-findings.md.
#
# Assembled by scripts/build-submission.py from the repository.

# ============================================================================
# SOURCE: remediate/workload-fixed.yaml
# ============================================================================
# accounts-api — remediated workload. Section 4a; defects A1–A8 in docs/04-findings.md.
# Threat T2 (pod identity / lateral movement), T4 (artefact integrity).
#
# This file is also the POSITIVE fixture for `kyverno test`: every policy in
# policy/kyverno/policies must admit it. If a policy change starts rejecting this
# manifest, the policy is wrong or this manifest needs to change with it — the
# test suite forces that decision to be made explicitly.
apiVersion: v1
kind: ServiceAccount
metadata:
  name: accounts-api
  namespace: accounts
  annotations:
    # IRSA. Role trust is scoped to this exact namespace/SA pair — see
terraform/iam_pod_identity.tf
    eks.amazonaws.com/role-arn: arn:aws:iam::123456789012:role/accounts-api-pod
automountServiceAccountToken: false     # A1: pods opt in individually, below
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: accounts-api
  namespace: accounts
  labels:
    app: accounts-api
    app.kubernetes.io/part-of: retail-banking
    data-classification: pii-and-cardholder-adjacent   # drives the Falco + log routing rules
spec:
  replicas: 3
  selector:
    matchLabels:
      app: accounts-api
  template:
    metadata:
      labels:
        app: accounts-api
        data-classification: pii-and-cardholder-adjacent
    spec:
      serviceAccountName: accounts-api
      automountServiceAccountToken: true   # A1: needed for IRSA; scoped by the Role below

      # A2: no host namespaces. Without this the NetworkPolicy in k8s/ is inert.
      hostNetwork: false
      hostPID: false
      hostIPC: false

      # A4: pod-level baseline. Restated per-container because a future container
```

```
      # added to this pod inherits the pod context but can override it.
      securityContext:
        runAsNonRoot: true
        runAsUser: 10001
        runAsGroup: 10001
        fsGroup: 10001
        seccompProfile:
          type: RuntimeDefault

      # A8: keep a bug in this service from becoming an outage for the other
      # twelve teams sharing the cluster.
      topologySpreadConstraints:
        - maxSkew: 1
          topologyKey: topology.kubernetes.io/zone
          whenUnsatisfiable: ScheduleAnyway
          labelSelector:
            matchLabels: { app: accounts-api }

      containers:
        - name: api
          # A6: digest, not `:latest`. The tag is retained in a comment for humans;
          # nothing resolves it. Signature over this digest is verified at admission.
          # tag at time of writing: 1.4.7-9f2c1ab77e04
          image: 210987654321.dkr.ecr.ap-south-1.amazonaws.com/accounts-
api@sha256:3f79b8e6b6e2c9d1c9f0a4a3e5d7c8b9a1f2e3d4c5b6a7988776655443322110
          imagePullPolicy: IfNotPresent

          securityContext:
            runAsNonRoot: true
            runAsUser: 10001                 # A4
            allowPrivilegeEscalation: false  # A4 — blocks setuid/file-capability escalation
            privileged: false
            readOnlyRootFilesystem: true     # A4 — attacker cannot drop tooling on disk
            capabilities:
              drop: ["ALL"]
            seccompProfile:
              type: RuntimeDefault

          # A5: no `envFrom: secretRef`. The DB credential arrives as a file from
          # the Secrets Store CSI driver (k8s/externalsecret.yaml) and the app
          # re-reads it on change, which is what makes a 5-minute emergency
          # rotation possible without a restart. Only non-sensitive config is env.
          env:
            - name: DB_CREDENTIAL_FILE
              value: /var/run/secrets/db/credential.json
            - name: AWS_REGION
              value: ap-south-1
            - name: OTEL_SERVICE_NAME
              value: accounts-api
            - name: LOG_REDACTION_MODE
              value: strict                  # see docs/06-log-redaction.md
          envFrom:
            - configMapRef:
                name: accounts-api-config    # non-sensitive only

          ports:
            - name: http
              containerPort: 8080

          # A8
          resources:
            requests: { cpu: "250m", memory: "256Mi" }
            limits:   { cpu: "1",    memory: "512Mi" }
```

```
          livenessProbe:
            httpGet: { path: /healthz, port: http }
            initialDelaySeconds: 10
          readinessProbe:
            httpGet: { path: /readyz, port: http }

          volumeMounts:
            # A3/A7: no hostPath. Writable scratch is an emptyDir, so
            # readOnlyRootFilesystem costs nothing operationally.
            - name: tmp
              mountPath: /tmp
            - name: db-credential
              mountPath: /var/run/secrets/db
              readOnly: true

      volumes:
        - name: tmp
          emptyDir: { medium: Memory, sizeLimit: 64Mi }
        - name: db-credential
          csi:
            driver: secrets-store.csi.k8s.io
            readOnly: true
            volumeAttributes:
              secretProviderClass: accounts-api-db
---
# A1: the RBAC rewrite. `verbs: ["*"]` on `secrets` became: read one named
# ConfigMap, and nothing else. In particular NO access to secrets at all — the
# database credential never transits the Kubernetes API, so a stolen SA token
# does not yield it. And no `create pods`, which was the escalation primitive.
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: accounts-api
  namespace: accounts
rules:
  - apiGroups: [""]
    resources: ["configmaps"]
    resourceNames: ["accounts-api-config"]   # named resource, not the whole type
    verbs: ["get", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: accounts-api
  namespace: accounts
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: Role
  name: accounts-api
subjects:
  - kind: ServiceAccount
    name: accounts-api
    namespace: accounts
```